

MongoDB

Laboratorio Práctico

CRUD · Consultas · Aggregation Pipeline · API externa

MongoDB 7.0

Node.js

`mongosh`

Open Library API

Bases de datos aplicada – NO OFICIAL



1. SQL vs NoSQL

Diferencias conceptuales y cuándo usar cada uno



2. Setup del laboratorio

Base de datos tienda_online con 4 colecciones reales



3. CRUD básico

Insert, Find, Update, Delete + comparativa con SQL



4. Consultas avanzadas

Operadores, arrays embebidos, regex, paginación



5. Aggregation Pipeline

Group, Lookup, Unwind, Facet y más



6. Consumo de API externa

Open Library API → MongoDB en tiempo real



7. Índices y performance

Cómo optimizar queries con explain()

SQL vs NoSQL — Conceptos clave

Entender las diferencias para elegir la herramienta correcta

Concepto	SQL / Relacional	MongoDB / Documental
Estructura	Tablas + filas fijas	Colecciones + documentos BSON
Schema	Rígido (DDL obligatorio)	Flexible (schema-less)
Relaciones	JOINS entre tablas	Documentos embebidos o \$lookup
Arrays	Tabla intermedia + JOIN	Array nativo en el documento
Escalabilidad	Vertical (más hardware)	Horizontal (sharding nativo)
Transacciones	ACID completo	ACID multi-documento (v4+)
Consultas	SQL estándar	MQL + Aggregation Pipeline

Dataset del Laboratorio — tienda_online

clientes

6 documentos

Campos:

nombre, email, plan,
saldo_fidelidad,
preferencias (objeto),
historial_pagos (array)

productos

8 documentos

Campos:

nombre, categoria, precio,
stock,
especificaciones (objeto),
tags (array)

pedidos

6 documentos

Campos:

numero, cliente_id, items
(array de obj),
subtotal, descuento, total,
envio (obj)

reseñas

4 documentos

Campos:

producto_id, cliente_id,
calificacion,
titulo, comentario, util

► `mongosh < 01_setup_datos.js` – Inserta todos los datos en segundos

NIVEL 1 — CRUD Básico

CREATE

SQL

```
INSERT INTO productos
(nombre, precio, stock)
VALUES ('Teclado', 89.99, 75)
```

MongoDB

```
db.productos.insertOne({
  nombre: 'Teclado',
  precio: 89.99,
  stock: 75
})
```

READ

SQL

```
SELECT nombre, precio
FROM productos
WHERE categoria = 'libros'
```

MongoDB

```
db.productos.find(
  { categoria: 'libros' },
  { nombre: 1, precio: 1 }
)
```

UPDATE

SQL

```
UPDATE productos
SET precio = precio * 1.1
WHERE nombre = 'Laptop'
```

MongoDB

```
db.productos.updateOne(
  { nombre: 'Laptop' },
  { $mul: { precio: 1.1 } }
)
```

DELETE

SQL

```
DELETE FROM productos
WHERE stock = 0
AND activo = false
```

MongoDB

```
db.productos.deleteMany({
  stock: 0,
  activo: false
})
```

Documentos embebidos

Lo que requiere 3 tablas en SQL, MongoDB lo hace en 1 documento

SQL — 3 tablas + 2 JOINS

SQL

```
clientes (id, nombre, email)
  ↓ JOIN
cliente_preferencias (cliente_id, categoria)
  ↓ JOIN
historial_pagos (cliente_id, metodo)

SELECT c.nombre, p.categoria, h.metodo
FROM clientes c
JOIN cliente_preferencias p ON c.id = p.cliente_id
JOIN historial_pagos h ON c.id = h.cliente_id
WHERE c.email = 'ana@email.com';
```



MongoDB — 1 documento, o JOINS

MongoDB

```
{
  nombre: "Ana García",
  email: "ana@email.com",
  preferencias: {
    categorias: ["electrónica","libros"],
    notificaciones: true
  },
  historial_pagos: ["tarjeta","transferencia"]
}

db.clientes.findOne({
  email: "ana@email.com"
});
```



1 sola lectura

Sin JOINS = latencia mínima



Dato atómico

El documento es la unidad de consistencia



Schema flexible

Cada cliente puede tener distintas preferencias

NIVEL 2 — Consultas Avanzadas

Filtros con operadores

```
db.productos.find({
  precio: { $gte: 100, $lte: 500 },
  stock: { $gt: 0 }
}).sort({ precio: 1 })
```

💡 *\$gte, \$lte, \$gt, \$lt, \$ne, \$in, \$nin*

Búsqueda en arrays

```
// Un tag
db.productos.find({ tags: "gaming" })

// Varios tags simultáneos
db.productos.find({
  tags: { $all: ["gaming","monitor"] }
})
```

💡 *Sin tabla intermedia — el array es nativo*

Campos anidados (dot notation)

```
db.clientes.find({
  plan: "premium",
  "preferencias.notificaciones": true
})
```

💡 *SQL requeriría JOIN a tabla de preferencias*

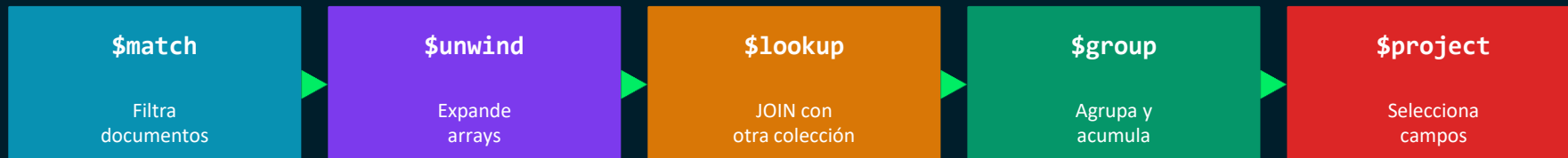
Arrays de objetos embebidos

```
// Pedidos con item de precio > $500
db.pedidos.find({
  "items.precio_unitario": { $gt: 500 }
})
```

💡 *SQL requeriría JOIN a tabla pedido_items*

NIVEL 3 — Aggregation pipeline

La herramienta más poderosa de MongoDB — procesa datos como un pipeline Unix



Ejercicio 3.2 — Ventas por categoría

```
db.pedidos.aggregate([
  { $match: { estado: "entregado" } },
  { $unwind: "$items" },
  {
    $lookup: {
      from: "productos",
      localField: "items.producto_id",
      foreignField: "_id",
      as: "producto_info"
    }
  },
  { $unwind: "$producto_info" },
  {
    $group: {
      _id: "$producto_info.categoria",
      unidades: { $sum: "$items.cantidad" },
      revenue: { $sum: { $multiply: ["$items.cantidad", "$items.precio_unitario"] } }
    }
  },
  { $sort: { revenue: -1 } }
])
```

Resultado:

Categoría	Unidades	Revenue
electrónica	4	\$3,279
hogar	1	\$449
libros	3	\$149

Superpoderes — \$facet y \$bucket

\$facet — 3 análisis en 1 query

```
// $facet: múltiples aggregations EN UNA SOLA QUERY
db.productos.aggregate([
  {
    $facet: {
      // Análisis por categoría
      por_categoria: [
        { $group: { _id: "$categoria", cantidad: { $sum: 1 } } }
      ],
      // Top 3 más vendidos
      mas_vendidos: [
        { $sort: { total_vendidos: -1 } }, { $limit: 3 },
        { $project: { nombre: 1, total_vendidos: 1 } }
      ],
      // Estadísticas generales
      estadisticas: [
        { $group: {
          _id: null,
          precio_min: { $min: "$precio" },
          precio_max: { $max: "$precio" },
          precio_prom: { $avg: "$precio" }
        } }
      ]
    }
  }
])
```

\$bucket — histograma

```
// $bucket: histograma de precios
db.productos.aggregate([
  {
    $bucket: {
      groupBy: "$precio",
      boundaries: [0, 50, 100, 300, 700, 1500],
      default: "1500+",
      output: {
        cantidad: { $sum: 1 },
        nombres: { $push: "$nombre" }
      }
    }
  }
])
// Resultado: segmentos de precios con
// cantidad de productos en cada rango
```

💡 En SQL esto requeriría múltiples CTEs o queries separados — en MongoDB es una sola llamada al servidor

BONUS — Consumo de API Externa



BONUS — Consumo de API Externa

03_api_externa.js

```
const url = "https://openlibrary.org/search.json?subject=mongodb&limit=8";
const response = await fetch(url);
const data = await response.json();

// Mapear al schema de nuestra tienda
const productos = data.docs.map(libro => ({
  nombre: libro.title,
  categoria: "libros",
  precio: +(Math.random() * 40 + 25).toFixed(2),
  especificaciones: {
    autor: libro.author_name?.join(", "),
    anio: libro.first_publish_year,
    isbn: libro.isbn?.[0]
  },
  calificacion_promedio: +libro.ratings_average?.toFixed(1) || 4.0,
  importado_api: true
}));

await db.collection("productos").insertMany(productos);
// → 8 libros reales de MongoDB insertados ✅
```

Índices y Performance

Ejemplo con explain()

```
// Sin índice → COLLSCAN (lee TODOS los docs)
db.pedidos.find({ estado: "entregado" }).explain("executionStats")
// totalDocsExamined: 6 ← toda la colección

// Crear índice
db.pedidos.createIndex({ estado: 1 })
db.clientes.createIndex({ email: 1 }, { unique: true })
db.productos.createIndex({ categoria: 1, precio: 1 }) // índice compuesto
db.productos.createIndex({ tags: 1 }) // índice en array

// Con índice → IXSCAN ✅
db.pedidos.find({ estado: "entregado" }).explain("executionStats")
// totalDocsExamined: 3 ← solo los relevantes
// executionTimeMillis: <1
```

📌 Reglas de oro para índices:

- Creá índices en los campos que más filtrás (WHERE/match)
- Los índices aceleran lecturas pero ralentizan escrituras
- Usá explain() para diagnosticar queries lentos

Simple

```
{ campo: 1 }
```

Un campo, ascendente (1) o descendente (-1)

Compuesto

```
{ cat: 1, precio: -1 }
```

Múltiples campos — orden importa

Único

```
{ email: 1 }, { unique: true }
```

Garantiza valores únicos

En array

```
{ tags: 1 }
```

Indexa cada elemento del array

Texto

```
{ nombre: "text" }
```

Búsqueda full-text

TTL

```
{ fecha: 1 }, {
  expireAfterSeconds: 3600 }
```

Auto-expira documentos

Resumen del Laboratorio

NIVEL 1

CRUD Básico

- 1.1 insertOne() — nuevo producto
- 1.2 find() + proyección
- 1.3 updateOne() con \$mul
- 1.4 updateMany() masivo
- 1.5 deleteMany() condicional

NIVEL 2

Consultas Avanzadas

- 2.1 Operadores de comparación
- 2.2 Arrays con \$all y \$in
- 2.3 Dot notation anidado
- 2.4 \$or / \$and combinados
- 2.5 Regex en texto
- 2.6 Arrays de objetos
- 2.7 sort + limit + skip

NIVEL 3

Aggregation

- 3.1 \$group básico (ventas)
- 3.2 \$lookup + \$unwind
- 3.3 \$bucket (histograma)
- 3.4 Análisis de clientes
- 3.5 \$facet (multi-pipeline)
- 3.6 \$addFields + \$map

BONUS

API + Performance

- Open Library API (fetch)
- Mapeo JSON → MongoDB
- insertMany() importación
- Índices: createIndex()
- explain() diagnóstico

¡Gracias!

MongoDB — De SQL a NoSQL con confianza



Docs oficiales:

mongodb.com/docs



MongoDB University:

learn.mongodb.com (gratis)



Open Library API:

openlibrary.org/developers



Código del lab:

01_setup_datos.js | 02_ejercicios_consultas.js |
03_api_externa.js



MongoDB — Build with the data platform that scales with you.